Приложение тональность комментариев

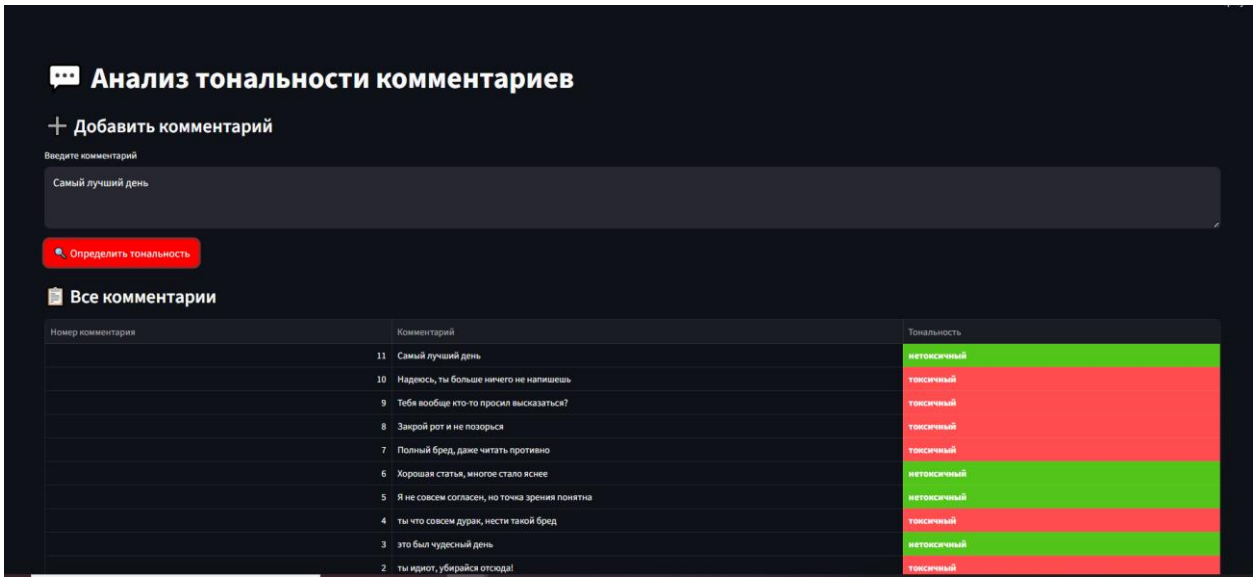
Часть сервера api (main.py):

```
from fastapi import FastAPI
from pydantic import BaseModel
import tensorflow as tf
import numpy as np
import pickle
import uvicorn

app = FastAPI(title="Toxic Comment API")
MAX_LEN = 128

model = tf.keras.models.load_model("gru_toxic_model.keras")


with open("tokenizer.pkl", "rb") as f:
    tokenizer = pickle.load(f)

class TextRequest(BaseModel):
    text: str


@app.post("/predict_text")
def predict_text(data: TextRequest):
    text = data.text
    seq = tokenizer.texts_to_sequences([text])

    padded = tf.keras.preprocessing.sequence.pad_sequences(
        seq, maxlen=MAX_LEN
    )

    prediction = model.predict(padded)[0][0]

    sentiment = "токсичный" if prediction >= 0.5 else "нетоксичный"

    return {
        "sentiment": sentiment,
        "confidence": float(prediction)
    }

@app.get("/health")
def health():
    return {"status": "ok"}

if __name__ == "__main__":
    uvicorn.run(app, host="localhost", port=8008)
```

Часть клиента (streamlit.py)

```
import streamlit as st
import requests
import pandas as pd
import mysql.connector

def get_db_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="1234",
```

```
        database="comments",
        port=3306
    )

def predict_sentiment(text: str):
    try:
        response = requests.post(
            "http://localhost:8008/predict_text",
            json={"text": text},
        )
        response.raise_for_status()
        return response.json()
    except Exception as e:
        st.error(f"Ошибка API: {e}")
        return None


st.set_page_config(page_title="Анализ комментариев", layout="wide")
st.title("💬 Анализ тональности комментариев")

st.subheader("➕ Добавить комментарий")

new_comment = st.text_area(
    "Введите комментарий",
    height=120,
    placeholder="Напишите комментарий..."
)

if st.button("🔍 Определить тональность", type="primary"):
    if not new_comment.strip():
        st.warning("Комментарий не может быть пустым")
    else:
        result = predict_sentiment(new_comment)

        if result is None:
            st.stop()

        sentiment = result["sentiment"]
        confidence = result["confidence"]

        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute(
            """
            INSERT INTO comment (text_comment, tonality_text)
            VALUES (%s, %s)
            """,
            (new_comment, sentiment)
        )
        conn.commit()
        cursor.close()
        conn.close()

        st.rerun()

st.subheader("📋 Все комментарии")

conn = get_db_connection()
df = pd.read_sql(
    "SELECT id_comment, text_comment, tonality_text FROM comment ORDER BY
id_comment DESC",
    conn
)
```

```
conn.close()


df = df.rename(columns={
    "id_comment": "Номер комментария",
    "text_comment": "Комментарий",
    "tonality_text": "Тональность"
})

def highlight_tonality(val):
    if val == "токсичный":
        return (
            "background-color: #ff4d4f;"
            "color: white;"
            "font-weight: bold;"
            "text-align: center;"
        )
    return (
        "background-color: #52c41a;"
        "color: white;"
        "font-weight: bold;"
        "text-align: center;"
    )

st.dataframe(
    df.style
      .applymap(highlight_tonality, subset=["Тональность"])
      .set_properties(**{
          "font-size": "16px",
          "padding": "8px"
      }),
    use_container_width=True,
    hide_index=True
)
```

Результат: